PROJECT REVIEW / JAVA WEB APPLICATION

Personal Finance Manager

A practical workspace for tracking expenses, planning budgets, and keeping savings goals visible.

JAVA 11+ · SERVLETS + JSP · TOMCAT 9 · MYSQL 8

REVIEW DECK · 02 OCTOBER 2026

Everyday money, in one focused workspace

Three connected jobs keep personal finance activity organized.

01 / EXPENSES

Track spending

Record dated expenses with a category, amount, and optional description. Review or remove entries from your own list.

02 / BUDGETS

Plan with budgets

Set a category limit for a monthly or yearly period, then keep budget totals visible alongside spending.

03 / GOALS

Save toward goals

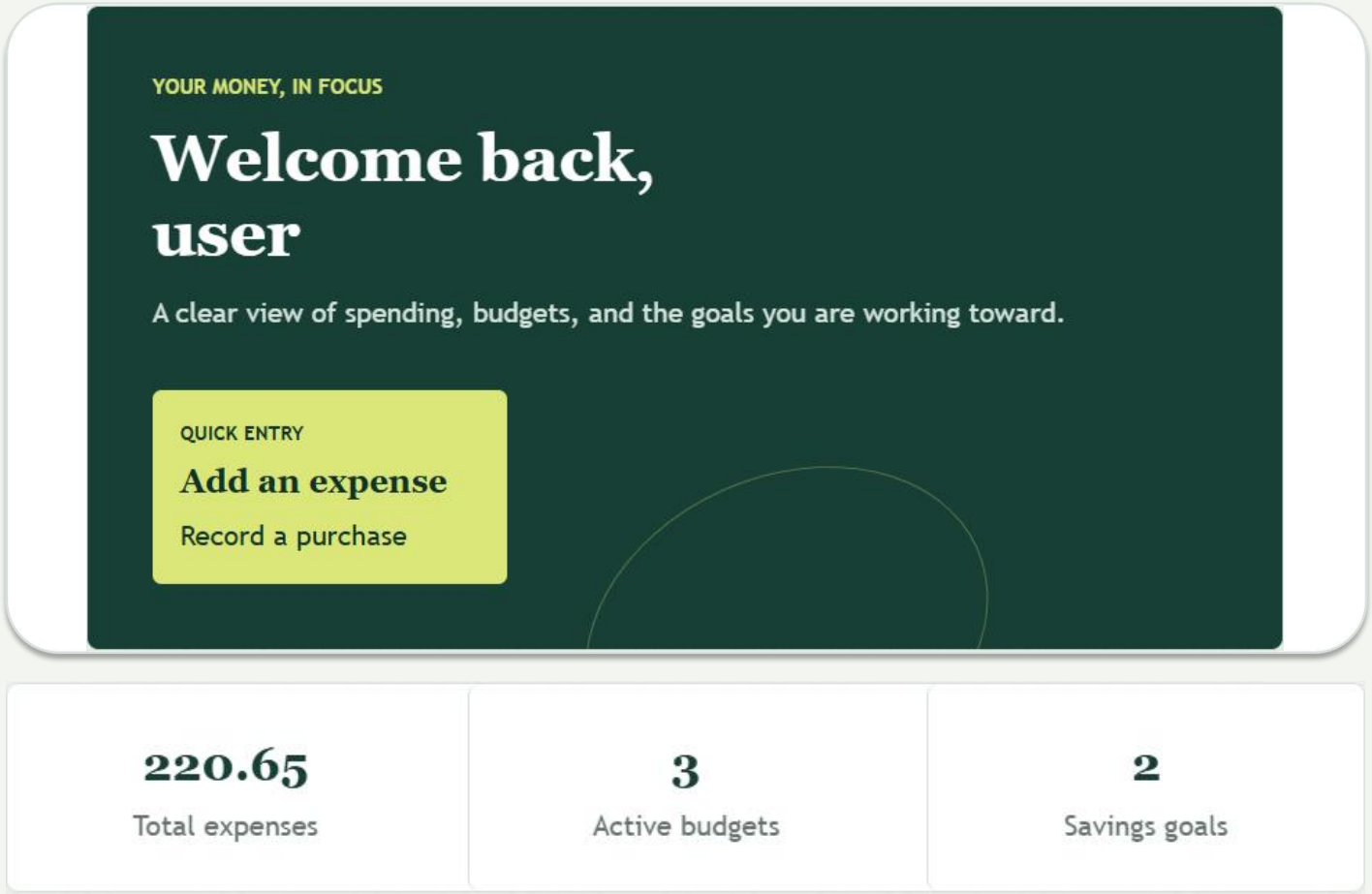
Give a savings target a name, amount, and deadline so future priorities have a place in the plan.

CAPTURE → **PLAN** → **REVIEW**

One account → personal records → a dashboard that points to the next useful action

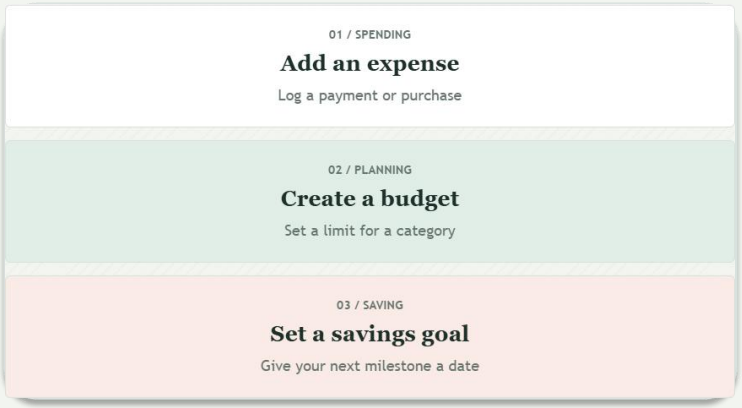
Dashboard, from welcome to next action

A composed view of three genuine dashboard regions from the running app.



KPI VALUES USE TEMPORARY REVIEW DATA

QUICK ACTIONS



Visible role

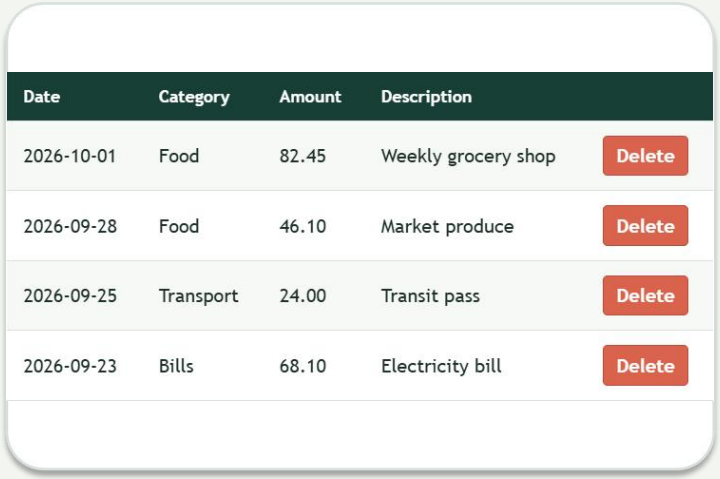
The signed-in account and USER/ADMIN role remain easy to spot.

Linked totals

Expenses, budgets, and goals open their detail lists.

The detail pages make the records useful

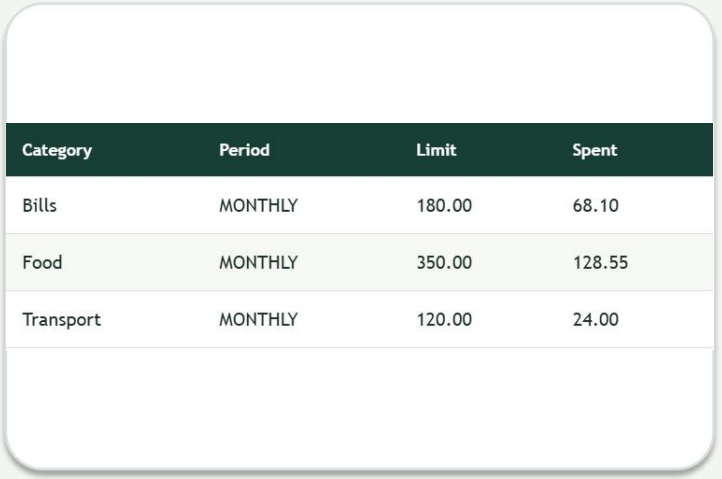
Forms and lists keep entry simple and make saved data easy to review.



Date	Category	Amount	Description	
2026-10-01	Food	82.45	Weekly grocery shop	Delete
2026-09-28	Food	46.10	Market produce	Delete
2026-09-25	Transport	24.00	Transit pass	Delete
2026-09-23	Bills	68.10	Electricity bill	Delete

Expenses

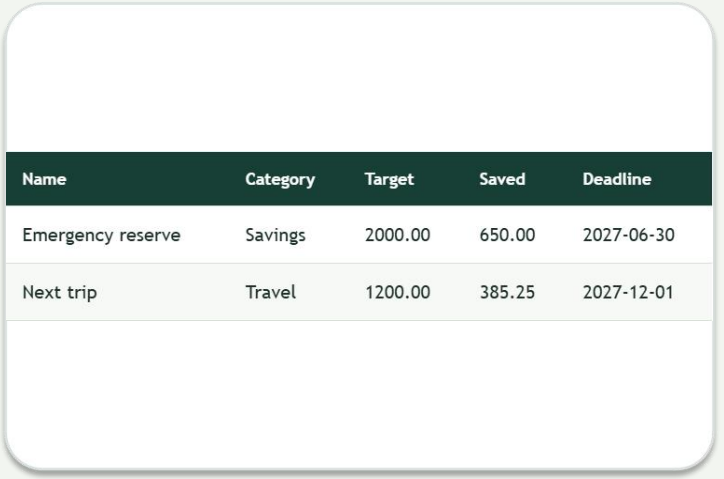
Date · category · amount · note



Category	Period	Limit	Spent
Bills	MONTHLY	180.00	68.10
Food	MONTHLY	350.00	128.55
Transport	MONTHLY	120.00	24.00

Budgets

Period limits and stored spend



Name	Category	Target	Saved	Deadline
Emergency reserve	Savings	2000.00	650.00	2027-06-30
Next trip	Travel	1200.00	385.25	2027-12-01

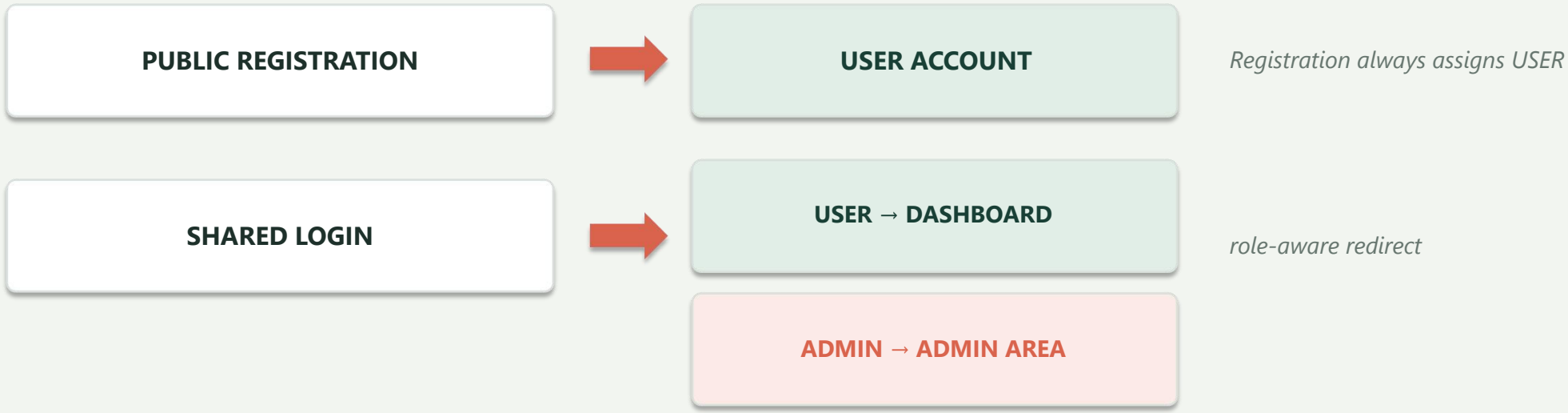
Goals

Targets · progress · deadline

The current user sees only their own records; admin reporting is a separate, role-guarded route.

Registration is open; admin creation is not

Role is assigned by the server and checked again on protected routes.



ADMIN-ONLY /admin

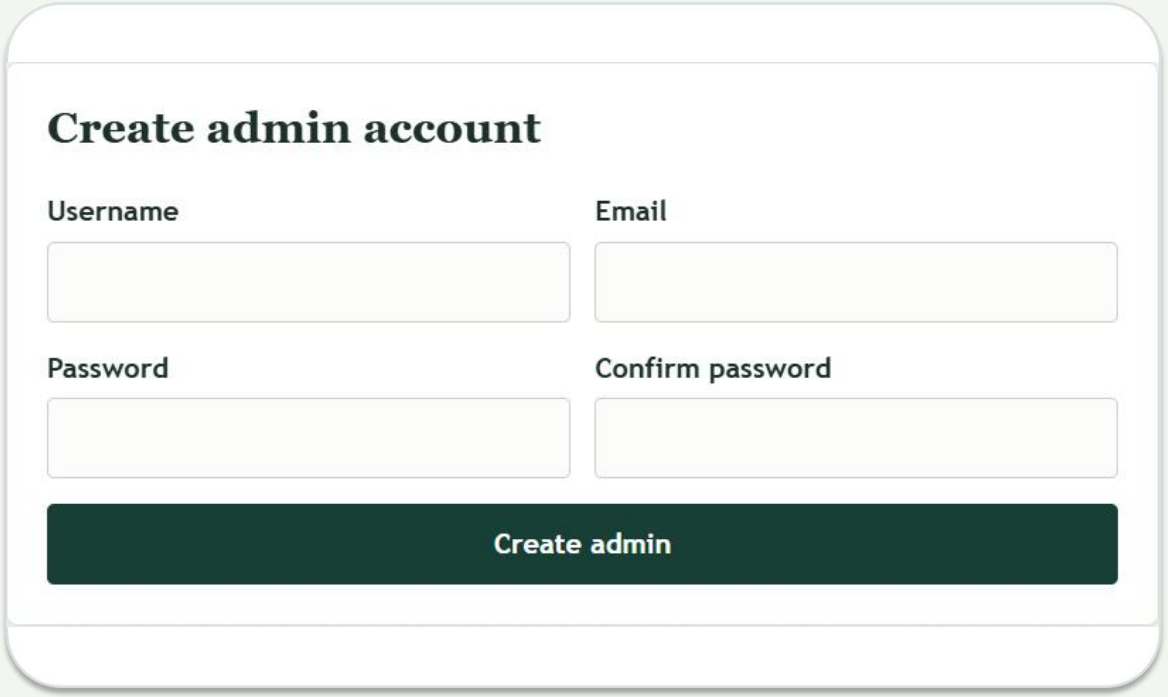
Create admin accounts · inspect cross-user records · reject non-admin sessions

USER AREA

Manage personal expenses, budgets, goals, and password

Admin tools are protected, not public

The admin route requires an authenticated ADMIN session before it can show the account form or records.



SCREENSHOT CROPPED TO ACCOUNT-CREATION PANEL; USER EMAILS OMITTED

Create admin accounts

Only an existing administrator can create another admin login.

Review all records

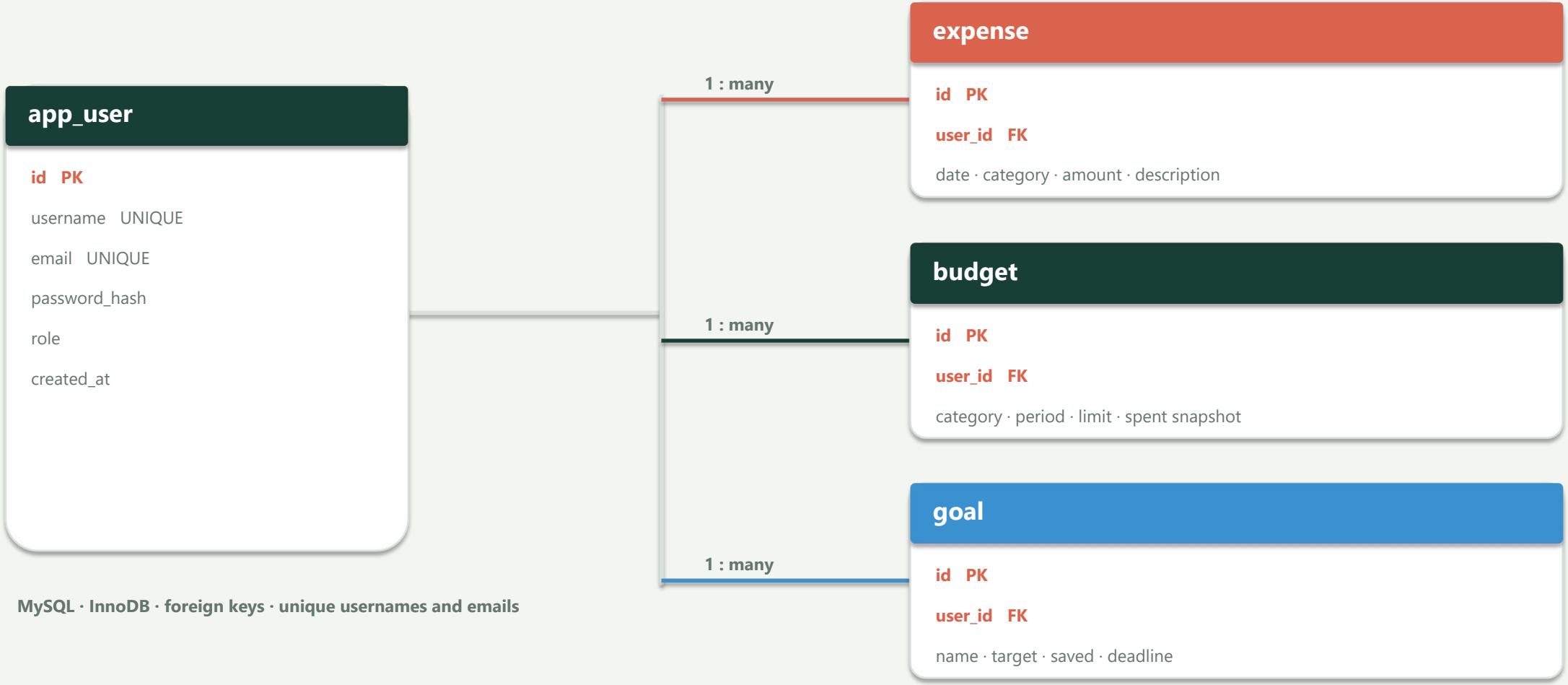
The admin view lists users and cross-user finance records read-only.

Server-side guard

Unauthenticated visitors go to login; non-admin users receive 403.

One account anchors three record types

Every finance record carries a user_id foreign key; personal queries filter by the signed-in owner.



MySQL · InnoDB · foreign keys · unique usernames and emails

A compact MVC-style Java web stack

The WAR packages application code, JSP views, and runtime dependencies for Tomcat.



Java 11+ · Maven WAR · javax.servlet 4 · JSP/JSTL · JDBC + HikariCP

Login dispatches by role; controllers forward to JSPs under /views.

The important boundaries are explicit

Security comes from server-side checks and data-scoped queries—not hidden UI controls.

AUTHORIZATION

Role checks on protected routes

AdminServlet checks the signed-in session role before showing data or creating admins.

CREDENTIALS

BCrypt password storage

PasswordUtil hashes passwords; password change checks the current password and requires 8+ characters.

DATA ISOLATION

Owner-scoped operations

Expense lookup, update, and delete repository methods include both record ID and user ID.

OPERATIONS

Environment-based DB config

FINANCE_DB_URL, FINANCE_DB_USER, and FINANCE_DB_PASSWORD stay outside source code.

DATA NOTE / budget.spent_amount is stored and incremented when an expense is added.

Expense edits/deletes are not reconciled yet, so this value can drift; add recomputation and tests next.

Build the WAR, deploy to Tomcat, open the app

The repository includes setup instructions in README.md and a manual evaluator walkthrough in TESTING.md.

01

Prepare

Java 11+ · Maven
MySQL 8 · Tomcat 9

02

Create schema

Run src/main/resources/db/schema.sql
Set FINANCE_DB_* for Tomcat

03

Package

mvn clean package
Build target/personal-finance-web.war

04

Deploy

Copy WAR to Tomcat webapps
Open /personal-finance-web/

DATABASE SETUP

Schema seeds demo roles for local review.

Change demo passwords before public use; keep real DB secrets in environment variables.

A clear walkthrough; coverage can keep growing

Seven focused unit tests now cover password hashing and authentication behavior.

BUILD CHECK

PASS

mvn clean package

WAR artifact: target/personal-finance-web.war

Build verified for this review deck on 02 Oct 2026.

- **Create a regular account**

  Public registration assigns USER; log in and review the dashboard.

- **Exercise the core workflows**

  Add an expense, budget, and goal; verify lists and dashboard counts.

- **Check access boundaries**

  After logout, request protected routes; try cross-user record IDs.

Next: repository/DB integration tests and data-isolation coverage. The manual walkthrough remains in TESTING.md.

Planned ownership across the team

A proposed division of responsibility; update the slide as each contribution is completed.

SB

Sachin Bhati

JAVA BACKEND & SECURITY

Authentication, role guards, password changes, and service behavior.

STRETCH: reconcile budgets after expense edits/deletes; add regression tests.

AB

Aayush Bagauli

DATA & INTEGRATION

MySQL schema, JDBC persistence and owner scoping; Tomcat/Maven setup and verification.

Own the end-to-end checklist, setup notes, and review presentation.

AP

Ayush Pandey

FRONTEND & UX

JSP/CSS screens, responsive forms and tables, dashboard workflows, and accessibility checks.

Capture and review the primary user and admin flows at desktop and mobile sizes.

PLANNED RESPONSIBILITIES · Confirm task handoffs as work is completed.

TAKEAWAY

A small finance app, with clear next steps.

Track everyday records. Keep personal data scoped. Give reviewers a predictable path through the product.

REVIEW PATH

Register as USER
↓
Log a transaction
↓
Plan a budget or goal
↓
Review as ADMIN

NEXT QUALITY STEP

Expand coverage to repository/database integration tests, then repeat the documented data-isolation checks.